บทที่ 5

ภาวะติดตาย

ในระบบปฏิบัติการที่ทำงานแบบหลายโปรแกรม (Multiprogramming) จะมีโพรเซสจำนวนมากทำงานพร้อมกัน และแต่ละโพรเซสมักต้องร้องขอทรัพยากรเพื่อให้การทำงานดำเนินต่อไปได้ ทรัพยากรเหล่านี้อาจเป็นหน่วยความจำ พื้นที่ดิสก์ เครื่องพิมพ์ หรือแม้แต่อุปกรณ์นำเข้า-ส่งออกอื่น ๆ ซึ่งล้วนมีจำนวนจำกัด เมื่อหลายโพรเซสร้องขอใช้งานทรัพยากรในเวลาใกล้เคียงกัน โอกาสที่จะเกิดการแข่งขันและการรอคอยจึงเป็นสิ่งที่หลีกเลี่ยงไม่ได้ หากระบบไม่มีวิธีการจัดการที่เหมาะสม อาจเกิดภาวะที่โพรเซสทั้งหมดไม่สามารถดำเนินงานต่อได้เพราะรอคอยกันและกันอยู่ในลักษณะเป็นวงจร

สถานการณ์นี้เรียกว่าภาวะติดตาย ซึ่งถือเป็นหนึ่งในปัญหาสำคัญของระบบปฏิบัติการ เพราะเมื่อเกิดขึ้นแล้วจะทำให้ไม่มีโพรเซสใดสามารถเสร็จสิ้นการทำงานได้ อีกทั้งยังทำให้ทรัพยากรถูกกักเก็บไว้โดยไม่ได้ใช้งานอย่างมีประสิทธิภาพ การทำความเข้าใจภาวะติดตายจึงมีความสำคัญ เนื่องจากจะช่วยให้เราสามารถออกแบบกลไกการป้องกัน การหลีกเลี่ยง หรือการกู้คืนระบบจากภาวะนี้ได้อย่างเหมาะสม และยังช่วยให้นิสิตมีมุมมองที่ชัดเจนยิ่งขึ้นว่าการจัดการทรัพยากรของระบบปฏิบัติการนั้นมีความซับซ้อนและท้าทายเพียงใด

5.1 การทำงานหลายโปรแกรมและภาวะติดตาย

ในการทำงานแบบหลายโปรแกรม มักมีโพรเซสหลายตัวทำงานพร้อมกัน ซึ่งโพรเซสเหล่านี้ต้องการใช้งานทรัพยากรของระบบที่มีอยู่อย่างจำกัด เช่น หน่วยประมวลผลกลาง (CPU) หน่วยความจำหลัก (Memory) หรือพื้นที่เก็บข้อมูล (Disk) เป็นต้น เมื่อโพรเซสร้องขอทรัพยากร หากทรัพยากรว่าง ระบบจะจัดสรรให้ทันที แต่หากทรัพยากรถูกใช้งานอยู่ โพรเซสนั้นจะต้องรอจนกว่าจะมีการคืนทรัพยากร อย่างไรก็ตาม อาจเกิดสถานการณ์ที่โพรเซสต้องรออย่างไม่มีที่สิ้นสุด เนื่องจากทรัพยากรที่ต้องการถูกครอบครองโดยโพรเซสอื่น และโพรเซสนั้นเองก็กำลังรอทรัพยากรอีกชนิดหนึ่งที่ยังไม่ว่างเช่นกัน ทำให้ทั้งสองฝ่ายไม่สามารถดำเนินงานต่อได้ เหตุการณ์เช่นนี้เรียกว่า**ภาวะติดตาย (Deadlock)**

5.1.1 ขั้นตอนการจัดการทรัพยากรของโพรเซส

เมื่อโพรเซสต้องการใช้ทรัพยากรของระบบ จะต้องปฏิบัติตามลำดับขั้นตอนต่าง ๆ ดังนี้

1. **การร้องขอ (Request)** – โพรเซสจะส่งคำร้องขอไปยังระบบ หากทรัพยากรว่าง ระบบจะจัดสรรให้ทันที แต่หากทรัพยากรถูกใช้งานอยู่ โพรเซสจะต้องรอ

2. **การใช้งาน (Use)** – เมื่อได้รับทรัพยากรแล้ว โพรเซสสามารถใช้งานได้ เช่น การใช้เครื่องพิมพ์เพื่อพิมพ์ข้อมูลออกมา

3. **การคืน (Release)** – เมื่อใช้งานเสร็จ โพรเซสต้องคืนทรัพยากรกลับสู่ระบบ เพื่อให้โพรเซสอื่นสามารถนำไปใช้ต่อได้

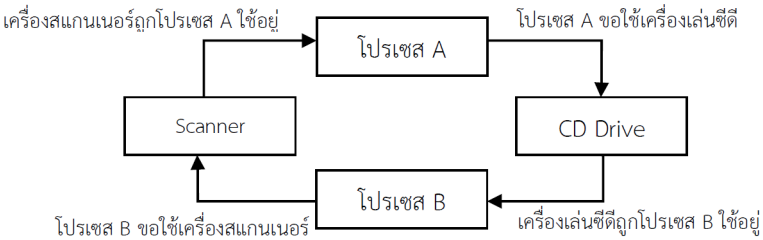
5.1.2 ตัวอย่างภาวะติดตาย

ให้พิจารณาตัวอย่างโพรเซสสองตัวคือ A และ B

- โพรเซส A ร้องขอใช้**เครื่องสแกนเนอร์ (Scanner)** และได้รับสิทธิ์ใช้งาน
- โพรเซส B ร้องขอใช้**เครื่องเขียนแผ่นซีดี (CD writer)** และได้รับสิทธิ์ใช้งาน

และในเวลาต่อมา โพรเซส A ต้องการใช้เครื่องเขียนแผ่นซีดี แต่ไม่สามารถใช้งานได้จนกว่าโพรเซส B จะคืนทรัพยากร ในขณะเดียวกัน โพรเซส B ก็ร้องขอใช้เครื่องสแกนเนอร์ แต่ไม่สามารถใช้งานได้เพราะโพรเซส A กำลังครอบครองอยู่

ดังนั้น โพรเซสทั้งสองต่างรอคอยซึ่งกันและกันโดยไม่มีทางสิ้นสุด ส่งผลให้เกิดเหตุการณ์**ภาวะติดตาย** ดังแสดงในรูปที่ 5.1



รูปที่ 5.1: โพรเซสสองโพรเซสเกิดภาวะติดตาย

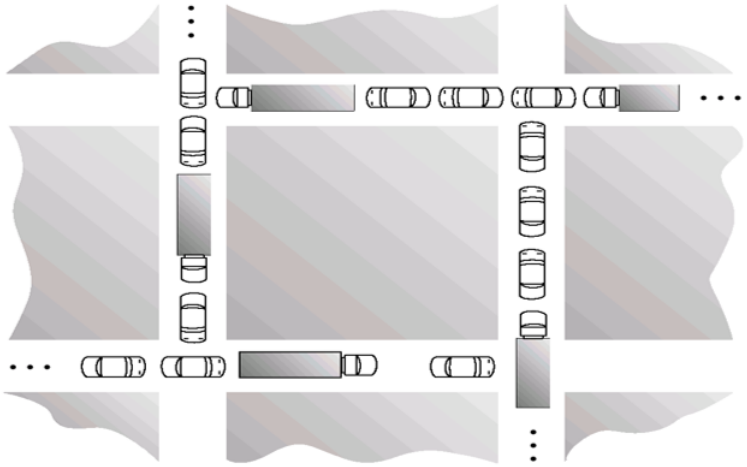
ภาวะติดตายไม่จำเป็นต้องเกิดขึ้นภายในคอมพิวเตอร์เพียงเครื่องเดียวเท่านั้น แต่สามารถเกิดขึ้นในระบบที่เชื่อมต่อกันหลายเครื่องได้เช่นกัน ตัวอย่างเช่น อุปกรณ์ที่ถูกใช้งานร่วมกันผ่านเครือข่ายสำนักงาน (Network) ซึ่งผู้ใช้หลายคนสามารถร้องขอใช้งานพร้อมกันได้ กรณีนี้หากมีการร้องขอและการรอคอยทรัพยากรเกิดขึ้นในลักษณะวนรอบ ก็อาจนำไปสู่ภาวะติดตายได้เช่นเดียวกัน

5.1.3 ตัวอย่างการจราจรในสี่แยก

ตัวอย่างที่เข้าใจง่ายของภาวะติดตายแสดงไว้ในรูปที่ 5.2 โดยสมมุติสถานการณ์การจราจรที่มีรถยนต์มาจาก 4 ทิศทางเข้าสู่สี่แยกพร้อมกัน เราสามารถเปรียบเส้นทางแต่ละทิศเป็นทรัพยากรของระบบ และรถยนต์แต่ละคันเป็นโพรเซสที่ต้องการใช้ทรัพยากรนั้นเพื่อเคลื่อนที่ต่อไปยังเส้นทางตรงข้าม

ตามปกติแล้ว กฎการขับรถในสี่แยกจะกำหนดให้รถที่อยู่ทางด้านขวามือมีสิทธิ์ไปก่อน กฎนี้สามารถทำงานได้อย่างราบรื่นในกรณีที่มีรถเข้ามาพร้อมกันเพียง 2 หรือ 3 คัน เช่น หากมีรถที่ต้องการไปทิศเหนือและรถอีกคันต้องการไปทิศตะวันตก รถที่จะไปทิศเหนือจะต้องรอให้รถทิศตะวันตกผ่านไปก่อน

อย่างไรก็ตาม หากมีรถทั้ง 4 คันมาถึงพร้อมกัน แต่ละคันจะต้องรอรถที่อยู่ทางขวามือก่อน ตามกฎที่กำหนดไว้ ทำให้เกิดการรอคอยวนซ้ำโดยไม่มีที่สิ้นสุด สถานการณ์นี้จึงเป็น**ภาวะติดตาย**ของการจราจร อีก



รูปที่ 5.2: ภาวะติดตายของการจราจรบริเวณสี่แยก

กรณีหนึ่งคือ หากรถทั้งสี่คันไม่ทำตามกฎดังกล่าว แต่ต่างคนต่างวิ่งเข้ามาในสี่แยกและจองเส้นทางไปคนละทิศ ผลลัพธ์คือรถทุกคันไม่สามารถเคลื่อนต่อไปได้ เพราะเส้นทางที่แต่ละคันต้องการถูกจองโดยรถคันอื่นแล้ว ซึ่งก็นำไปสู่ภาวะติดตายเช่นเดียวกัน

5.2 คุณลักษณะของภาวะติดตาย

ภาวะติดตาย เป็นสถานการณ์ที่ไม่พึงประสงค์ในระบบ เพราะเมื่อเกิดขึ้นแล้วจะทำให้ไม่มีโพรเซสใดสามารถทำงานจนเสร็จสิ้นได้ และทรัพยากรของระบบจะถูกครอบครองโดยโพรเซสที่ติดอยู่ในภาวะดังกล่าว ดังนั้นก่อนที่จะศึกษาวิธีการจัดการกับภาวะติดตาย เราจำเป็นต้องเข้าใจถึง**เงื่อนไขที่ทำให้เกิดภาวะติดตาย** เสียก่อน ภาวะติดตายจะเกิดขึ้นได้ก็ต่อเมื่อเงื่อนไขทั้งสี่ข้อต่อไปนี้เกิดขึ้นพร้อมกัน คือ

1. **การไม่เกิดร่วม (Mutual Exclusion)** มีทรัพยากรอย่างน้อยหนึ่งชนิดที่ไม่สามารถใช้งานร่วมกันได้ (non-sharable) กล่าวคือ ทรัพยากรดังกล่าวจะถูกครอบครองได้เพียงโพรเซสเดียวในเวลาใดเวลาหนึ่งเท่านั้น

2. **ถือและรอ (Hold and Wait)** มีโพรเซสที่ครอบครองทรัพยากรบางส่วนอยู่ และในขณะเดียวกันยังร้องขอทรัพยากรเพิ่มเติม ซึ่งทรัพยากรที่ร้องขอนั้นกำลังถูกใช้งานโดยโพรเซสอื่น

3. **ไม่มีการบังคับ (No Preemption)** ทรัพยากรที่ถูกครอบครองไม่สามารถถูกบังคับให้คืนได้ จะถูกปล่อยออกมาก็ต่อเมื่อโพรเซสที่ครอบครองเสร็จสิ้นการใช้งานเท่านั้น

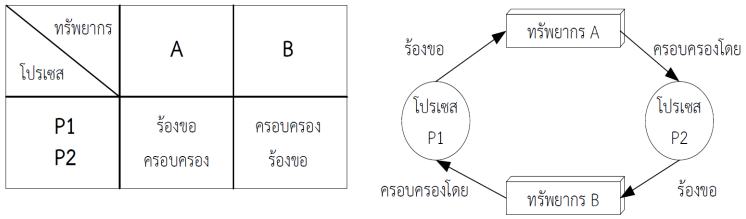
4. **การรอแบบวงกลม (Circular Wait)** ต้องมีเซตของโพรเซส $\{P_0, P_1, \ldots, P_n\}$ ที่อยู่ในภาวะรอคอยวนรอบ กล่าวคือ P_0 รอทรัพยากรที่ถูกถือโดย P_1, P_1 รอทรัพยากรที่ถูกถือโดย P_2 และดำเนินเช่นนี้ไปเรื่อย ๆ จนถึง P_{n-1} รอทรัพยากรที่ถือโดย P_n และ P_n รอทรัพยากรที่ถือโดย P_0

5.2.1 การวิเคราะห์เงื่อนไข

ในบรรดาเงื่อนไขทั้งสี่ข้อ เงื่อนไขสามข้อแรก (*Mutual Exclusion, Hold and Wait, No Preemption*) เป็นเพียง**เงื่อนไขจำเป็น** แต่ยังไม่เพียงพอที่จะทำให้เกิดภาวะติดตาย ส่วนเงื่อนไขข้อที่สี่ (*Circular Wait*)

ถือเป็นผลลัพธ์ที่เกิดขึ้นเมื่อสามเงื่อนไขแรกเกิดขึ้นพร้อมกัน ซึ่งจะนำไปสู่การรอคอยวนรอบแบบไม่มีที่สิ้นสุดได้

ดังนั้น ภาวะติดตายจะเกิดขึ้นได้ก็ต่อเมื่อ **เงื่อนไขทั้งสี่ข้อครบถ้วนสมบูรณ์** ดังแสดงในรูปที่ 5.3



รูปที่ 5.3: ภาวะลูกโซ่ของโพรเซสที่ต่างรอคอยซึ่งกันและกัน

5.3 แบบจำลองระบบ

เพื่อให้สามารถอธิบายและจัดการภาวะติดตายได้อย่างเป็นระบบ เราสามารถแสดงรายละเอียดของความสัมพันธ์ระหว่างโพรเซสและทรัพยากรในรูปแบบของกราฟ

ทรัพยากรในระบบสามารถแบ่งออกเป็นหลายชนิด เช่น หน่วยประมวลผลกลาง (CPU cycle), พื้นที่หน่วยความจำ (Memory space), อุปกรณ์นำเข้า/ส่งออก (I/O device) เป็นต้น โดยแต่ละทรัพยากร R_i จะมีจำนวนที่คงที่ W_i ซึ่งสามารถถูกใช้งานโดยโพรเซสต่าง ๆ ได้ตามลำดับขั้นตอน ดังนี้

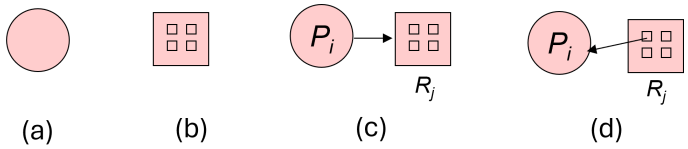
- **ร้องขอ (Request)**: โพรเซสส่งคำร้องขอเพื่อใช้งานทรัพยากร
- **ใช้งาน (Use)**: โพรเซสใช้งานทรัพยากรที่ได้รับการจัดสรร
- **ปลดปล่อย (Release)**: เมื่อใช้งานเสร็จ โพรเซสคืนทรัพยากรให้แก่ระบบ

5.3.1 กราฟการจัดสรรทรัพยากร

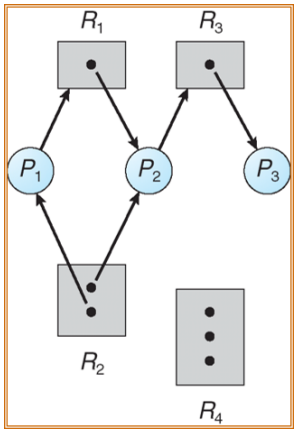
กราฟการจัดสรรทรัพยากร (Resource-Allocation Graph: RAG) ใช้แทนความสัมพันธ์ระหว่างโพรเซสและทรัพยากรในระบบ โดยประกอบด้วย

- เซตของจุดยอด (Vertices) V แบ่งออกเป็น 2 ประเภท ได้แก่
  - $P = \{P_1, P_2, \ldots, P_n\}$: เซตของโพรเซสทั้งหมด
  - $R = \{R_1, R_2, \ldots, R_m\}$: เซตของทรัพยากรทั้งหมด
- เซตของเส้นเชื่อม (Edges) E ซึ่งแบ่งออกเป็น 2 ประเภท
  - **Request edge** : $P_i \rightarrow R_j$ หมายถึง โพรเซส P_i กำลังร้องขอทรัพยากร R_j และอยู่ในสถานะรอ
  - **Assignment edge** : $R_j \rightarrow P_i$ หมายถึง ทรัพยากร R_j ถูกจัดสรรให้กับโพรเซส P_i

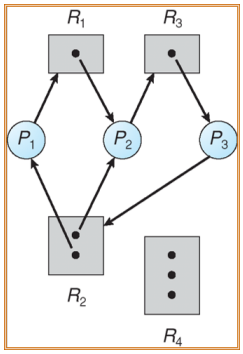
ดังแสดงในรูปที่ 5.4 ถึงรูปที่ 5.7



รูปที่ 5.4: กราฟการจัดสรรทรัพยากร: (a) โพรเซส (b) ทรัพยากรที่มี 4 หน่วย (c) P_i ร้องขอ R_j (d) P_i กำลังถือ R_j



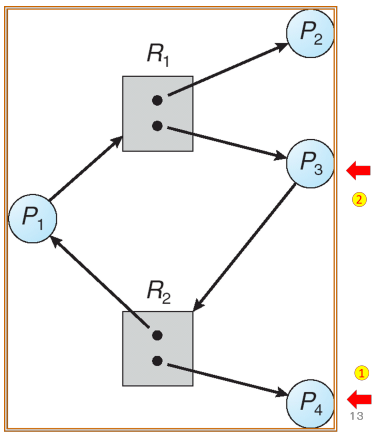
รูปที่ 5.5: ตัวอย่างกราฟการจัดสรรทรัพยากรทั่วไป



รูปที่ 5.6: ตัวอย่างกราฟการจัดสรรทรัพยากรที่เกิดภาวะติดตาย

ข้อสังเกต:

- หากกราฟการจัดสรรทรัพยากร**ไม่มีวงกลม (cycle)** แสดงว่าระบบจะไม่เกิดภาวะติดตาย
- หากกราฟการจัดสรรทรัพยากร**มีวงกลม (cycle)** ระบบอาจเกิดภาวะติดตายได้ ขึ้นอยู่กับจำนวนหน่วยของทรัพยากร
  - ถ้าทรัพยากรแต่ละชนิดมีเพียง 1 หน่วย: จะเกิดภาวะติดตายแน่นอน
  - ถ้าทรัพยากรมีหลายหน่วย: ระบบอาจไม่เกิดภาวะติดตายก็ได้



รูปที่ 5.7: ตัวอย่างกราฟการจัดสรรทรัพยากรที่ไม่เกิดภาวะติดตาย (แม้จะมีการเชื่อมต่อแบบวงกลม)

5.4 วิธีการจัดการกับภาวะติดตาย

การจัดการกับปัญหาภาวะติดตายของระบบปฏิบัติการ สามารถทำได้หลายวิธี ซึ่งโดยทั่วไปแบ่งออกเป็น 4 วิธีหลัก ดังนี้

1. **การป้องกัน (Prevention)** ใช้วิธีการต่าง ๆ เพื่อรับประกันว่าเงื่อนไขของภาวะติดตายจะไม่เกิดขึ้นในระบบ
2. **การหลีกเลี่ยง (Avoidance)** ใช้วิธีการจัดสรรทรัพยากรอย่างรอบคอบและมีแบบแผน เพื่อหลีกเลี่ยงไม่ให้ระบบเข้าสู่ภาวะติดตาย
3. **การกู้คืน (Recovery)** ยอมให้ระบบสามารถเกิดภาวะติดตายได้ แต่เมื่อเกิดขึ้นแล้วจะใช้กลไกการตรวจสอบและกู้คืนเพื่อทำให้ระบบกลับมาทำงานต่อได้
4. **การปล่อยผ่าน (Ignore)** ไม่จัดการกับภาวะติดตายโดยตรง และถือว่าภาวะติดตายจะไม่เกิดขึ้น วิธีนี้ถูกนำมาใช้ในระบบปฏิบัติการส่วนใหญ่ รวมทั้ง *Unix* สำหรับระบบ *Java Virtual Machine (JVM)* ก็ไม่มีการจัดการภาวะติดตายเช่นกัน เนื่องจากยกหน้าที่ให้ผู้พัฒนาแอปพลิเคชันเป็นผู้ดูแลเอง

5.4.1 การป้องกันภาวะติดตาย

การป้องกันภาวะติดตาย (*Deadlock Prevention*) คือการทำให้เงื่อนไขทั้งสี่ข้อที่จำเป็นต่อการเกิดภาวะติดตายไม่สามารถเกิดขึ้นพร้อมกันได้ ซึ่งสามารถอธิบายได้ดังนี้

- **เงื่อนไขการไม่เกิดร่วม (Mutual Exclusion)** ไม่อนุญาตให้โพรเซสร้องขอทรัพยากรที่สามารถใช้งานร่วมกันได้ (*sharable resources*) ในลักษณะเฉพาะตัว โพรเซสจะถือครองได้เฉพาะทรัพยากรที่ไม่สามารถใช้ร่วมกัน (*non-sharable resources*) เท่านั้น
- **เงื่อนไขถือและรอ (Hold and Wait)** ต้องรับประกันว่า เมื่อโพรเซสร้องขอทรัพยากรใหม่ โพรเซสนั้นจะต้องไม่ถือทรัพยากรอื่นอยู่ในมือ วิธีการทำได้สองแนวทาง คือ
  1. ให้โพรเซสร้องขอทรัพยากรทั้งหมดที่ต้องการตั้งแต่เริ่มทำงาน
  2. อนุญาตให้โพรเซสร้องขอทรัพยากรใหม่ได้ก็ต่อเมื่อไม่ได้ครอบครองทรัพยากรใดอยู่เลย

ข้อเสียของวิธีนี้คือ ประสิทธิภาพการใช้ทรัพยากรอาจต่ำ และมีความเป็นไปได้ที่จะเกิดการรอคอยไม่รู้จบ

- **เงื่อนไขไม่มีการบังคับ (No Preemption)** หากโพรเซสที่ครอบครองทรัพยากรอยู่ร้องขอทรัพยากรอื่นเพิ่ม แต่ไม่สามารถจัดสรรให้ได้ในขณะนั้น โพรเซสจะต้องถูกบังคับให้คืนทรัพยากรทั้งหมดที่ถืออยู่ แล้วเริ่มกระบวนการร้องขอใหม่ตั้งแต่ต้น เช่นเดียวกับโพรเซสที่เข้ามาใหม่

- **เงื่อนไขการรอแบบวงกลม (Circular Wait)** ต้องมีการจัดเรียงลำดับทรัพยากรทั้งหมดในระบบ (*total ordering*) และโพรเซสจะต้องร้องขอทรัพยากรตามลำดับที่กำหนดไว้ เช่น หากโพรเซส P_1 ครอบครองทรัพยากร R_3, R_4, R_6, R_7 แล้วร้องขอทรัพยากร R_8 จะไม่เกิดการรอแบบวงกลม เพราะเป็นการร้องขอตามลำดับหมายเลขทรัพยากรจากน้อยไปมาก แต่หากโพรเซสต้องการทรัพยากรที่มีหมายเลขน้อยกว่า โพรเซสจะต้องคืนทรัพยากรทั้งหมดที่ถืออยู่ก่อน

5.4.2 การหลีกเลี่ยงภาวะติดตาย

จากวิธีการป้องกันภาวะติดตายในหัวข้อที่ผ่านมา แม้จะช่วยไม่ให้เกิดภาวะติดตาย แต่ก็อาจทำให้การใช้ทรัพยากรของระบบไม่มีประสิทธิภาพสูงสุด ดังนั้น จึงมีอีกแนวทางหนึ่งคือ **การหลีกเลี่ยงภาวะติดตาย (Deadlock Avoidance)** ซึ่งใช้วิธีตรวจสอบและจัดสรรทรัพยากรอย่างรอบคอบ เพื่อป้องกันไม่ให้ระบบเข้าสู่สถานะที่เสี่ยงต่อการเกิดภาวะติดตาย

ตัวอย่างเช่น หากระบบมี*ดิสก์* 1 ตัว และ*เครื่องพิมพ์* 1 เครื่อง โพรเซส P อาจร้องขอใช้งานดิสก์ก่อนแล้วจึงใช้เครื่องพิมพ์ ขณะที่โพรเซส Q อาจทำตรงกันข้ามคือใช้เครื่องพิมพ์ก่อนแล้วจึงใช้ดิสก์ ดังนั้นนอกจากการตรวจสอบว่าทรัพยากรว่างหรือไม่ ระบบยังต้องพิจารณาด้วยว่า**โพรเซสใดกำลังใช้ทรัพยากรใดอยู่ และโพรเซสอื่นควรร้องขอใช้งานตามลำดับใด** เพื่อหลีกเลี่ยงการเกิดการรอคอยแบบวนซ้ำไม่สิ้นสุด

แนวคิดหลักของการหลีกเลี่ยงภาวะติดตายคือ ระบบจะต้องทำงานอยู่ใน**สถานะปลอดภัย (Safe State)** ซึ่งหมายถึงสถานะที่สามารถจัดสรรทรัพยากรได้ โดยไม่ทำให้เกิดการรอคอยวนรอบแบบไม่มีที่สิ้นสุด

สถานะที่ปลอดภัย

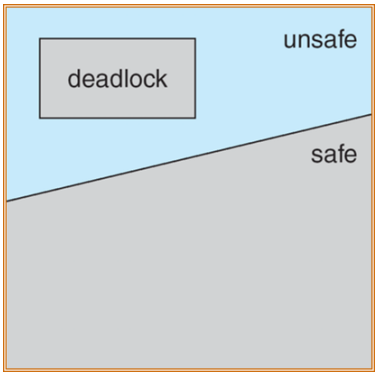
สถานะที่ปลอดภัย (Safe State) คือสถานะของระบบที่สามารถรับประกันได้ว่าการจัดสรรทรัพยากรจะไม่ก่อให้เกิดภาวะติดตาย เมื่อโพรเซสร้องขอทรัพยากรที่ว่าง ระบบจะ**ตรวจสอบ**ก่อนว่าหากจัดสรรไป ระบบจะยังคงอยู่ในสถานะปลอดภัยหรือไม่

หากการร้องขอสามารถเปลี่ยนจากเส้นโยง *ร้องขอ* ($P_i \rightarrow R_j$) ไปเป็นเส้นโยง*มอบหมาย* ($R_j \rightarrow P_i$) โดยที่ไม่ทำให้เกิดวงกลมในกราฟการจัดสรรทรัพยากร การจัดสรรนั้นจะได้รับอนุญาต แต่ถ้าทำให้เกิดวงกลมจะถือว่าระบบเข้าสู่**สถานะไม่ปลอดภัย (Unsafe State)**

ดังนั้น ขั้นตอนวิธีสำหรับการตรวจสอบจะพิจารณา**ว่ามีวงกลมในกราฟหรือไม่** ซึ่งมีความซับซ้อนเชิงเวลา $O(n^2)$ โดยที่ n คือจำนวนโพรเซสทั้งหมดในระบบ ถ้าไม่พบวงกลม แสดงว่าระบบยังอยู่ในสถานะปลอดภัย แต่ถ้าพบวงกลม ระบบอาจเข้าสู่ภาวะติดตายได้ ดังแสดงในรูปที่ 5.8

Resource-Allocation Graph Algorithm

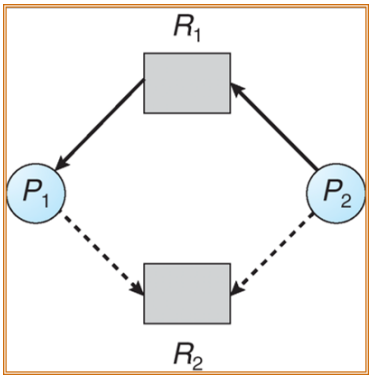
เทคนิควิธีที่นิยมใช้ในการหลีกเลี่ยงภาวะติดตาย คือการใช้**กราฟการจัดสรรทรัพยากร (Resource-Allocation Graph)** โดยมีการเพิ่ม*เส้นโยงเรียกร้อง (Claim Edge)* เข้าไปจากเดิมที่มีเพียงเส้นโยง*ร้องขอ (Request Edge)* และ*มอบหมาย (Assignment Edge)*



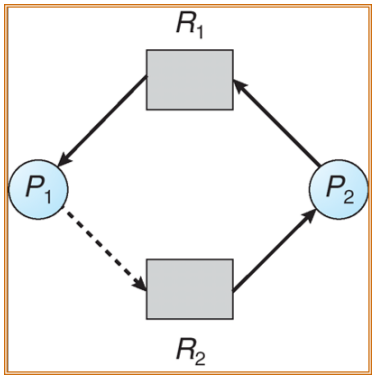
รูปที่ 5.8: Safe, Unsafe และ Deadlock State

- **เส้นโยงเรียกร้อง (Claim Edge)** : $P_i \dashrightarrow R_j$ (เส้นประ) หมายถึง โพรเซส P_i อาจร้องขอทรัพยากร R_j ในอนาคต
- เมื่อโพรเซส P_i ร้องขอใช้งานจริง **เส้นโยงเรียกร้อง** จะถูกแปลงเป็น **เส้นโยงร้องขอ**
- เมื่อโพรเซสได้รับการจัดสรรทรัพยากร **เส้นโยงร้องขอ** จะถูกแปลงเป็น **เส้นโยงมอบหมาย**
- เมื่อโพรเซสคืนทรัพยากร **เส้นโยงมอบหมาย** จะถูกแปลงกลับเป็น **เส้นโยงเรียกร้อง**

แนวทางนี้ทำให้ระบบสามารถตรวจสอบและคาดการณ์ได้ว่า การจัดสรรทรัพยากรใด ๆ จะนำไปสู่ภาวะติดตายหรือไม่ ดังแสดงในรูปที่ 5.9, 5.10, 5.11 และ 5.12



รูปที่ 5.9: กราฟการจัดสรรทรัพยากรสำหรับการหลีกเลี่ยงภาวะติดตาย



รูปที่ 5.10: ตัวอย่างสถานะไม่ปลอดภัยในกราฟการจัดสรรทรัพยากร

1. Let *Work* and *Finish* be vectors of length *m* and *n*, respectively. Initialize:
   $Work = Available$
   $Finish\ [i] = false$ for $i = 1,2,3, \ldots, n$.
2. Find an *i* such that both:
   (a) $Finish\ [i] = false$
   (b) $Need_i \leq Work$
   If no such *i* exists, go to step 4.
3. $Work = Work + Allocation_i$
   $Finish[i] = true$
   go to step 2.
4. If $Finish\ [i]$ == true for all *i*, then the system is in a safe state.

รูปที่ 5.11: Safety Algorithm (ใช้ตรวจสอบสถานะที่ปลอดภัย)

Request = request vector for process P_i. $Request_i[j] = k$ means process P_i wants *k* instances of resource type R_j.

1. If $Request_i \leq Need_i$ go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim.
2. If $Request_i \leq Available$, go to step 3. Otherwise P_i must wait, since resources are not available.
3. Pretend to allocate requested resources to P_i by modifying the state as follows:
   $Available = Available - Request_i$;
   $Allocation_i = Allocation_i + Request_i$;
   $Need_i = Need_i - Request_i$;
   - If safe $\Rightarrow$ the resources are allocated to P_i.
   - If unsafe $\Rightarrow$ P_i must wait, and the old resource-allocation state is restored

รูปที่ 5.12: Resource-Request Algorithm สำหรับโพรเซส P_i

ตัวอย่างที่ 1: Banker's Algorithm สมมติให้มี 5 โพรเซส P_0 ถึง P_4 และมีทรัพยากรอยู่ 3 ประเภท ได้แก่ A (10 หน่วย), B (5 หน่วย), และ C (7 หน่วย)

	Allocation	Max	Available
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

ตาราง Need คำนวณจาก Max – Allocation

	Need
	A B C
P_0	7 4 3
P_1	1 2 2
P_2	6 0 0
P_3	0 1 1
P_4	4 3 1

ระบบอยู่ใน **สถานะปลอดภัย (Safe State)** เพราะมีลำดับการทำงานที่เป็นไปได้คือ $< P_1, P_3, P_4, P_2, P_0 >$ ซึ่งสอดคล้องกับเกณฑ์ความปลอดภัย

ตัวอย่างที่ 2: Banker's Algorithm P_1 ร้องขอทรัพยากรเพิ่ม (1,0,2) โดยตรวจสอบว่า Request $\leq$ Available ในกรณีนี้ (1,0,2) $\leq$ (3,3,2) ซึ่งเป็นจริง

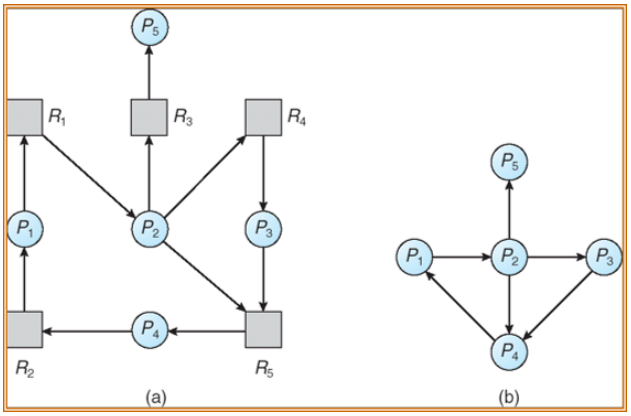
	Allocation	Need	Available
	A B C	A B C	A B C
P_0	0 1 0	7 4 3	$\underline{2}$ 3 $\underline{0}$
P_1	$\underline{3}$ 0 $\underline{2}$	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

เมื่อใช้ Safety Algorithm ตรวจสอบ พบว่าลำดับการทำงาน $< P_1, P_3, P_4, P_2, P_0 >$ ยังคงเป็นไปได้และระบบยังอยู่ในสถานะปลอดภัย

5.4.3 การตรวจหาภาวะติดตาย

การตรวจหาภาวะติดตาย (Deadlock Detection) เป็นแนวทางที่ยอมให้ระบบเข้าสู่สถานะที่มีภาวะติดตายได้ แล้วใช้ขั้นตอนวิธี (*Detection Algorithm*) เพื่อตรวจสอบว่ามีภาวะติดตายเกิดขึ้นหรือไม่ จากนั้นจึงใช้วิธีการกู้คืน (*Recovery Scheme*) เพื่อแก้ปัญหาและทำให้ระบบสามารถทำงานต่อได้

การตรวจหาภาวะติดตายมักอาศัย**กราฟการรอ (Wait-for Graph)** ซึ่งเป็นรูปแบบหนึ่งของกราฟการจัดสรรทรัพยากร โดยระบบจะคอยตรวจสอบว่ามี *วงกลม (cycle)*เกิดขึ้นในกราฟหรือไม่ หากมี แสดงว่ามีภาวะติดตายเกิดขึ้น ขั้นตอนวิธีนี้มีความซับซ้อนเชิงเวลา $O(n^2)$ โดยที่ n คือจำนวนโพรเซสในระบบ



รูปที่ 5.13: (a) กราฟการจัดสรรทรัพยากร (b) กราฟการรอที่สอดคล้องกัน

โครงสร้างที่ใช้ในการตรวจสอบประกอบด้วย:

- **Available**: เวกเตอร์ขนาด m แสดงจำนวนทรัพยากรแต่ละประเภทที่ยังเหลืออยู่
- **Allocation**: เมทริกซ์ขนาด $n \times m$ แสดงจำนวนทรัพยากรที่ถูกจัดสรรให้แต่ละโพรเซส
- **Request**: เมทริกซ์ขนาด $n \times m$ แสดงการร้องขอปัจจุบันของแต่ละโพรเซส เช่น หาก Request[i,j] = k หมายความว่าโพรเซส P_i กำลังร้องขอทรัพยากร R_j เพิ่มอีก k หน่วย

1. Let *Work* and *Finish* be vectors of length *m* and *n*, respectively Initialize:
   (a) *Work* = *Available*
   (b) For *i* = 1,2, …, *n*, if $Allocation_i \neq 0$, then
   Finish[i] = false; otherwise, *Finish*[i] = *true*.
2. Find an index *i* such that both:
   (a) $Finish[i] == false$
   (b) $Request_i \leq Work$
   If no such *i* exists, go to step 4.
3. $Work = Work + Allocation_i$
   $Finish[i] = true$
   go to step 2.
4. If $Finish[i] ==$ false, for some i, $1 \leq i \leq n$, then the system is in deadlock state. Moreover, if $Finish[i] == false$, then P_i is deadlocked.

รูปที่ 5.14: Detection Algorithm

ตัวอย่างที่ 3: การตรวจหาภาวะติดตาย สมมติให้มี 5 โพรเซส P_0 ถึง P_4 และมีทรัพยากร 3 ประเภท ได้แก่ A (7 หน่วย), B (2 หน่วย), และ C (6 หน่วย) โดย Snapshot ที่เวลา T_0:

	Allocation	Request	Available
	A B C	A B C	A B C
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

ลำดับการทำงาน $< P_0, P_2, P_3, P_1, P_4 >$ จะทำให้ Finish[i] = true สำหรับทุก i ได้
แต่ถ้าในกรณี P_2 ต้องการเพิ่มขึ้น:

	Request
	A B C
P_0	0 0 0
P_1	2 0 2
P_2	0 0 $\underline{1}$
P_3	1 0 0
P_4	0 0 2

ผลการตรวจสอบ: สามารถเรียกคืนทรัพยากรจาก P_0 ได้เท่านั้น แต่ยังไม่เพียงพอที่จะตอบสนองการร้องขอของโพรเซสอื่นทั้งหมด ดังนั้น ระบบจึงเกิด **ภาวะติดตาย (Deadlock)** ซึ่งเกี่ยวข้องกับโพรเซส P_1, P_2, P_3, และ P_4

เงื่อนไขการเลือกใช้อัลกอริทึมตรวจหา

การเลือกใช้อัลกอริทึมตรวจหาภาวะติดตาย ขึ้นอยู่กับปัจจัยดังต่อไปนี้

- ความถี่ของการเกิดภาวะติดตายในระบบ
- จำนวนโพรเซสที่จะได้รับผลกระทบหากเกิดภาวะติดตาย
- หากภาวะติดตายเกิดขึ้นบ่อย ขั้นตอนวิธีตรวจหาควรถูกเรียกใช้งานบ่อยตามไปด้วย เนื่องจากทรัพยากรที่ถูกจัดสรรให้กับโพรเซสในภาวะติดตายจะสูญเปล่า จนกว่าจะมีการกู้คืน อีกทั้งจำนวนโพรเซสที่เกี่ยวข้องกับภาวะติดตาย อาจเพิ่มขึ้นเรื่อย ๆ หากไม่ตรวจสอบและแก้ไขทันที

5.4.4 การกู้คืนจากภาวะติดตาย

เมื่อระบบเข้าสู่ภาวะติดตาย จะไม่สามารถทำงานใด ๆ ต่อไปได้ จึงจำเป็นต้องมีกลไกเพื่อกำจัดภาวะติดตายและกู้ระบบกลับคืนสู่สภาพที่ใกล้เคียงกับเดิม ทำให้สามารถดำเนินงานที่ยังค้างอยู่ต่อได้ กระบวนการนี้เรียกว่า **การกู้คืนภาวะติดตาย (Recovery from Deadlock)**

วิธีการกู้คืนภาวะติดตายสามารถทำได้ 2 วิธีหลัก คือ

- **การยกเลิกโพรเซส (Process Termination)**
- **การบังคับคืนทรัพยากร (Resource Preemption)**

การยกเลิกโพรเซส

วิธีนี้คือการยกเลิกโพรเซสที่เกี่ยวข้องกับภาวะติดตาย เพื่อให้สามารถเรียกคืนทรัพยากรและทำให้ระบบกลับมาทำงานได้ มีแนวทางดังนี้

1. **ยกเลิกทุกโพรเซสที่ติดตาย** เป็นวิธีที่ง่ายที่สุด โดยยกเลิกโพรเซสทั้งหมดที่อยู่ในวงจรติดตาย ข้อเสียคืออาจทำให้โพรเซสที่ทำงานใกล้เสร็จต้องถูกยกเลิก ซึ่งเสียทั้งเวลาและค่าใช้จ่ายเพราะต้องเริ่มทำงานใหม่ทั้งหมด

2. **ยกเลิกทีละโพรเซส** เป็นการยกเลิกโพรเซสทีละหนึ่ง จนกว่าภาวะติดตายจะหมดไป หลังจากยกเลิกแต่ละครั้ง ระบบต้องเรียกใช้อัลกอริทึมตรวจหาภาวะติดตายซ้ำ เพื่อตรวจสอบว่ายังมีการติดตายอยู่หรือไม่ ข้อเสียคือมีค่าใช้จ่ายสูง (overhead) จากการตรวจสอบซ้ำหลายครั้ง

ปัจจัยที่ใช้ในการพิจารณาเลือกโพรเซสที่จะถูกยกเลิก ได้แก่

- ระดับความสำคัญ (priority) ของโพรเซสนั้น
- เวลาที่โพรเซสทำงานมาแล้ว และเวลาที่ต้องใช้จนกว่างานจะเสร็จ
- จำนวนและประเภทของทรัพยากรที่โพรเซสกำลังใช้ และความยากง่ายในการคืนทรัพยากรเหล่านั้น
- ทรัพยากรเพิ่มเติมที่โพรเซสยังต้องการจนกว่างานจะเสร็จ
- จำนวนโพรเซสที่จำเป็นต้องถูกยกเลิกเพื่อแก้ไขภาวะติดตาย
- ประเภทของโพรเซส เช่น แบบโต้ตอบ (interactive) หรือแบบเป็นกลุ่ม (batch)

การบังคับคืนทรัพยากร

อีกวิธีคือการกู้คืนด้วยการ **แย่งชิงทรัพยากร (preemption)** จากโพรเซสที่ครอบครองอยู่ แล้วนำไปให้โพรเซสอื่นที่รอใช้งาน กระบวนการนี้ต้องพิจารณาปัจจัยหลายด้าน ดังนี้

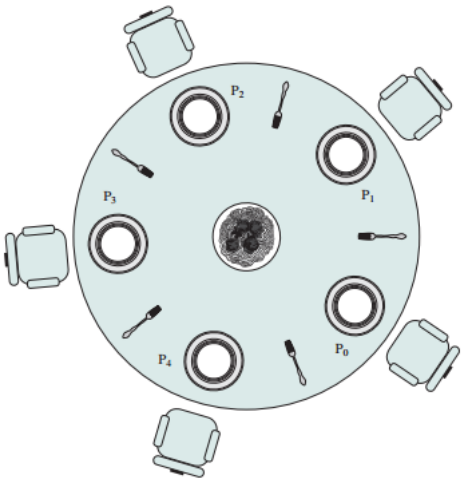
1. **การเลือกเหยื่อ (Selecting a Victim)** ระบบต้องตัดสินใจว่าจะยึดทรัพยากรใดจากโพรเซสใด โดยมักพิจารณาจากค่าใช้จ่าย (cost) ที่เกิดขึ้นหากยกเลิกโพรเซสนั้น เช่น เลือกโพรเซสที่ถือครองทรัพยากรน้อยที่สุด และทำงานไปแล้วไม่นาน

2. **การถอยกลับ (Rollback)** เมื่อโพรเซสถูกยึดทรัพยากรไป มันไม่สามารถดำเนินงานต่อได้ ระบบจึงต้องให้โพรเซสถอยกลับไปยังสถานะที่ปลอดภัย และเริ่มทำงานใหม่จากจุดนั้น วิธีที่ง่ายที่สุดคือให้ถอยกลับไปยังสถานะที่อยู่ไกลที่สุดจากการเกิดติดตาย ซึ่งจำเป็นต้องมีการจัดเก็บสถานะของโพรเซสไว้ล่วงหน้า

3. **การอดตาย (Starvation)** หากระบบเลือกเหยื่อโดยพิจารณาจากค่าใช้จ่ายเป็นหลัก อาจทำให้โพรเซสเดิมถูกแย่งชิงซ้ำ ๆ โดยไม่มีโอกาสได้ทำงานจนเสร็จ ซึ่งถือเป็นภาวะ**อดตาย (starvation)** ดังนั้นระบบต้องมีมาตรการป้องกันไม่ให้โพรเซสใดถูกละเลยตลอดไป

คำถามท้ายบท

1. ปัญหาอาหารเย็นของนักปราชญ์ (The Dining Philosophers Problem)



รูปที่ 5.15: ปัญหาอาหารเย็นของนักปราชญ์

2. ปัญหาผู้อ่านและผู้เขียน (The Readers/Writers Problem) ข้อมูลในระบบที่มีการทำงานของโพรเซสพร้อมกันหลายตัวจะถูกร่วมกันใช้งาน บางโพรเซสต้องการอ่านข้อมูล (Readers) ในอีกบางโพรเซสต้องการเขียนข้อมูล (Writer) โดยที่ความแตกต่างของทั้งสองส่วน คือ Readers จะเป็นโพรเซสที่ต้องการจะอ่านข้อมูลเพียงอย่างเดียว ส่วนโพรเซส Writer จะต้องการเพียงแค่การเขียนข้อมูลเท่านั้น สังเกตว่าหากมีหลายๆ โพรเซสทำการอ่านข้อมูลพร้อมๆ กันจะไม่มีปัญหาใดเกิดขึ้น แต่ถ้ามีโพรเซสที่ต้องการเขียนข้อมูลพร้อมๆ กันจะเกิดปัญหาขึ้นทันที